

คู่มือผู้ใช้โปรเจกต์ ai_web_scrpping

1. โปรเจกต์นี้ใช้ทำอะไร

ai_web_scrpping เป็น data pipeline สำหรับรวบรวมข้อมูลเชิงพื้นที่ของ 10 จังหวัดในไทย โดยเน้น landmark, POI, amenity และถนน แล้วแปลงเป็นไฟล์ CSV/JSON ที่นำไปใช้ต่อในงานวิเคราะห์, dashboard, BI หรือรายงานพื้นที่ได้ทันที

จุดสำคัญและสิ่งอำนวยความสะดวก

landmark, restaurant, amenity และ POI จาก OpenStreetMap/Google Maps

ถนนและโครงข่ายถนน

OSM roads, road intelligence และ Google road-name enrichment

บริบทพื้นที่

ประชากร อากาศ น้ำท่วม และ zone profile

ไฟล์ถนนปลายทาง

roads_final.csv สำหรับใช้ร่วมกับ landmark และรายงานพื้นที่

2. สิ่งที่ต้องเตรียมก่อนรัน

ให้รันทุกคำสั่งจาก root ของโปรเจกต์:

```
cd C:\ai_web_scrpping
.\venv\Scripts\Activate.ps1
```

ตรวจไฟล์ .env ว่ามี key ที่จำเป็น:

```
BOT_API_KEY=...
GOOGLE_MAPS_API_KEY=...
```

- BOT_API_KEY ใช้กับข้อมูลสินเชื่อจาก Bank of Thailand
- GOOGLE_MAPS_API_KEY ใช้กับ Google road-name enrichment
- ห้าม commit .env เพราะมี secret

3. โครงสร้างไฟล์ที่ควรรู้

data/raw/	ข้อมูลดิบและ cache จาก scraper
data/processed/	ไฟล์ CSV ที่พร้อมนำไปใช้ต่อ
scrapers/	ตัวดึงข้อมูลจากแหล่งต่าง ๆ
utils/	ตัว clean, merge, analyze, validate

กฎสำคัญ: อยากรันไฟล์ใน scrapers/ ตรง ๆ ให้ใช้ entrypoint ที่ root เช่น python main.py หรือ python run_roads_only.py

คำสั่งรัน Pipeline หลัก

1. Full Pipeline

```
python main.py
```

เมื่อรันแล้วจะเจอเมนูหลัก:

- [1] รันทุกอย่าง (Full Pipeline)
- [2] ดึงเฉพาะ Landmarks (OSM + Google Maps Sync)
- [3] ดึงเฉพาะ Restaurants
- [4] เลือกเองทีละรายการ (Custom)
- [q] ออกจากโปรแกรม

กด 1 เพื่อรันครบชุด, กด 2 เพื่ออัปเดตเฉพาะ landmark/POI, กด 3 เพื่อดึงร้านอาหาร, กด 4 เพื่อเลือก module เอง และ กด q เพื่อออก

ลำดับโดยรวม: ดึงข้อมูลลิ้นเชื่อ, landmarks และ POI, sync Google Maps, clean/merge, ตรวจ quality gate และ upload ไป S3

ข้อควรระวัง: ใน main.py ยังมี bucket placeholder your-target-bucket-name ถ้ายังไม่ได้ตั้ง S3 จริง อย่าใช้ flow upload เป็น production

2. โค้ดและฟังก์ชันสำคัญที่ควรรู้

- `main.py::main()`: จุดเริ่มต้นของเมนูหลักและตัวเลือก pipeline ทั้งหมด
- `main.py::run_landmarks_pipeline()`: รัน landmark จาก OSM แล้ว sync รายละเอียดจาก Google Maps
- `scrapers/landmarks.py::scrape_landmarks()`: ดึง landmark/POI ตาม layer และจังหวัดเป้าหมาย
- `scrapers/google_maps_sync.py::scrape_google_maps_sync()`: ดึงข้อมูล Google Maps กลับเข้า `landmarks_raw.json`
- `scrapers/road_analysis.py::fetch_province_roads()`: ดึงข้อมูลถนนจาก OSM/Overpass แล้วสร้าง `roads_raw.json` และ `roads.csv`
- `utils/google_road_enrichment.py::build_google_road_enrichment_outputs()`: ดึงชื่อถนนจาก Google และสร้าง `roads_enriched.csv/roads_final.csv`
- `utils/road_intelligence.py::build_road_intelligence_outputs()`: สร้าง feature, summary และ intersection ของถนน
- `utils/pipeline_quality.py::validate_pipeline_outputs()`: ตรวจสอบคุณภาพ output ก่อนถือว่า pipeline สำเร็จ

Road Dataset และ Google Enrichment

1. Road Dataset

```
python run_roads_only.py
```

ผลลัพธ์หลัก:

```
data/raw/roads_raw.json  
data/raw/roads_raw_quality_report.json  
data/processed/roads.csv
```

สร้าง road intelligence แยกได้ด้วยคำสั่งนี้:

```
python run_road_intelligence.py
```

ได้ roads_features.csv, roads_summary_by_province.csv, road_density_by_zone.csv,
road_intersections.csv

2. Google Road Name Enrichment

```
python run_google_road_enrichment.py --sleep-seconds 0.1 --cache-flush-interval 25
```

ทดสอบโดยไม่ยิง API:

```
python run_google_road_enrichment.py --dry-run --limit 20
```

สร้าง output จาก cache เดิมโดยไม่ยิง API เพิ่ม:

```
python run_google_road_enrichment.py --limit 0 --sleep-seconds 0
```

ไฟล์ Output สำคัญ

1. Landmark และ Amenity

```
data/processed/landmarks_clean.csv
data/processed/amenities_clean.csv
```

ใช้วิเคราะห์จุดสำคัญในพื้นที่ เช่น โรงพยาบาล ห้าง ตลาด สถานที่ราชการ ร้านบริการ และสิ่งอำนวยความสะดวกอื่น ๆ

- province, district, sub_district: พื้นที่ตั้ง
- name, name_en: ชื่อสถานที่
- category: ประเภทสถานที่
- layer: ระดับความสำคัญของ POI
- lat, lon: พิกัด

2. Road Output

```
data/processed/roads.csv
data/processed/roads_enriched.csv
data/processed/roads_final.csv
```

- roads.csv: ข้อมูลถนนดิบจาก OSM แบบ flatten เป็น CSV
- roads_enriched.csv: เพิ่มรายละเอียดจาก Google reverse geocode เพื่อ audit
- roads_final.csv: ไฟล์ถนนปลายทางที่ตัด Google detail ออกแล้ว ใช้ง่าย

3. คอลัมน์ที่ควรใช้ใน roads_final.csv

- road_name: ชื่อจาก OSM หรือ unnamed:<osm_id>
- road_ref: เลขทางหลวง/เลขถนน ถ้ามี
- road_display_name: ชื่อสำหรับแสดงผลจริง
- google_match_status: สถานะ enrichment เช่น matched, no_route, skipped_has_ref
- length_km: ความยาวของ OSM way โดยรวมระยะตาม geometry ของถนน

4. Cache ที่ไม่ควรลบทิ้งง่าย ๆ

```
data/raw/google_road_name_cache.json
data/raw/temp_roads_topology/
data/raw/geocode_cache.json
```

ไฟล์เหล่านี้ช่วยลดเวลารันซ้ำและลด API quota โดยเฉพาะ Google road enrichment

Troubleshooting หลัก

1. Google API ขึ้น REQUEST_DENIED

- เปิด Billing ใน Google Cloud Project แล้วหรือยัง
- เปิด Geocoding API แล้วหรือยัง
- API key restriction ตรงกับการใช้งานหรือไม่

สำหรับ script Python ควรใช้ Application restriction เป็น IP addresses ถ้าเครื่องมี public IP คงที่ และจำกัด API restriction เฉพาะ Geocoding API

2. Playwright เปิด browser ไม่ได้

```
python -m playwright install chromium
```

ใช้เมื่อตัว scraper ที่ต้องเปิด browser หา Chromium ไม่เจอ

3. Pipeline หยุดเพราะ Quality Gate

ถ้า quality gate fail ให้ดู error แล้วตรวจไฟล์เหล่านี้ก่อน:

```
data/processed/landmarks_clean.csv
data/processed/zone_profiles.csv
```

4. เมื่อ output หายหรือแถวไม่ครบ

ให้ตรวจจำนวนแถวจากไฟล์ต้นทางและไฟล์ปลายทางก่อนเสมอ:

```
(Import-Csv data\processed\roads.csv | Measure-Object).Count
(Import-Csv data\processed\roads_final.csv | Measure-Object).Count
```

ถ้า roads_enriched.csv ถูกเขียนไม่ครบ ให้ regenerate จาก cache โดยไม่ยิง API เพิ่ม:

```
python run_google_road_enrichment.py --limit 0 --sleep-seconds 0
```

จากนั้นค่อย export roads_final.csv ใหม่

การตรวจผลข้อมูลถนน

1. ข้อมูลถนนไม่มีชื่อ

ถนนที่ไม่มีชื่อจาก OSM จะอยู่ในรูป unnamed:<osm_id> อย่าทับ road_name โดยตรง เพราะใช้ trace กลับ OSM ได้ ให้ใช้ road_display_name สำหรับ dashboard/report แทน

2. สถานะที่พบบ่อย

- matched: เดิมชื่อจาก Google ได้
- no_route: Google ไม่คืนชื่อถนนที่ใช้ได้
- skipped_has_ref: ไม่มีชื่อ แต่มี road_ref จึง fallback เป็น Road <ref>
- skipped_named: มีชื่อจาก OSM อยู่แล้ว

3. วิธีเช็กผลเร็ว ๆ

ดูจำนวนแถวแยกตามสถานะ:

```
Import-Csv data\processed\roads_final.csv |
Group-Object google_match_status |
Select-Object Name,Count
```

ดูตัวอย่างชื่อถนน:

```
Import-Csv data\processed\roads_final.csv |
Select-Object -First 20 province,road_name,road_display_name
```

ดูเฉพาะแถวที่ยังไม่มีชื่อถนนที่ใช้ได้:

```
Import-Csv data\processed\roads_final.csv |
Where-Object { $_.google_match_status -eq "no_route" } |
Select-Object province,osm_id,highway_type,road_display_name
```

4. ไฟล์ที่ควรใช้ต่อ

```
data/processed/roads_final.csv
```

ถ้าต้อง audit ว่า Google คืนอะไรมา ให้เปิด data/processed/roads_enriched.csv